

Understanding LangChain Document Loaders

From raw files to retrievable chunks - the ingestion half of a RAG system

Course handout • agent-to-production • Session reference

1. Introduction

Every retrieval-augmented application starts with the same unglamorous problem: the knowledge you want the model to use lives in PDFs, web pages, CSV exports, Notion databases and ticket systems - not in the model's weights. A **document loader** is the LangChain component that turns one of those sources into a list of Document objects that the rest of the pipeline understands.

Loaders look trivial in a demo (two lines and you have text) and are the most common source of quality problems in production. Garbled PDF columns, missing page numbers, silently skipped files and over-large chunks all show up much later as "the bot hallucinates" or "retrieval returns nothing useful". This handout covers the loader API, the choices that actually matter, and the production concerns that follow from them.

What you should be able to do after this section:

- Load PDFs, directories, CSV/JSON and web pages into Document objects.
- Explain the difference between `load()`, `lazy_load()` and `aload()`, and pick the right one.
- Choose a chunk size and overlap deliberately instead of copying the defaults.
- Preserve metadata well enough to cite sources in an answer.
- Handle corrupt files, scanned pages and re-ingestion without re-embedding everything.

This file is also the demo document used in the session. It is deliberately multi-page and mixes headings, prose, tables and code so that chunking and retrieval behaviour is visible when you load it.

2. Where loaders sit in the pipeline

Retrieval-augmented generation is two pipelines, not one. The **ingestion** pipeline runs offline, usually on a schedule; the **query** pipeline runs per request. Loaders are step one of ingestion, and every later step inherits their mistakes.

INGESTION (offline, batch)

```
Source files ---> Load ---> Split ---> Embed ---> Store
pdf/csv/web   loader   splitter   model   vector DB
```

QUERY (online, per request)

```
Question ---> Embed ---> Search ---> Re-rank ---> Prompt ---> LLM ---> Answer
                                   (top-k chunks + their metadata become the context)
```

Two consequences worth stating early:

- **Loading is not parsing-neutral.** Two PDF loaders on the same file give different text, so they give different chunks, different embeddings and different answers. Treat the loader as a model choice, not plumbing.
- **Metadata is the only bridge back to the source.** If the loader drops the page number, no later step can invent it, and citations become impossible.

3. The Document object

Everything a loader produces is a Document: a piece of text plus a free-form metadata dictionary. That is the entire contract.

```
from langchain_core.documents import Document

doc = Document(
    page_content="LangChain provides document loaders that ingest data ...",
    metadata={
        "source": "reference-docs/langchain_demo.pdf", # set by the loader
        "page": 0, # 0-indexed for PyPDFLoader
        "team": "platform", # anything you add yourself
    },
)

print(len(doc.page_content)) # characters, not tokens
print(doc.metadata["page"])
```

A few facts that surprise people:

- **One document is not one file.** PyPDFLoader returns one document *per page*; CSVLoader returns one *per row*; TextLoader returns one for the whole file. Always check `len(docs)` before assuming.
- **Metadata keys are not standardised** beyond source. Different loaders emit different keys; normalise them yourself if you mix sources.
- **Metadata must stay filterable.** Most vector stores only support scalar values (str, int, float, bool). Nested dicts or lists are commonly rejected or silently dropped - flatten them at ingestion time.

4. The loader catalogue

LangChain ships hundreds of loaders. In practice a handful cover most work. Loaders live in `langchain_community.document_loaders` and most need one extra dependency.

Loader	Source	Granularity	Extra install
PyPDFLoader	PDF files	one doc per page	pypdf
PyMuPDFLoader	PDF files (fast, good layout)	one doc per page	pymupdf
UnstructuredPDFLoader	PDF incl. tables/images	element or page	unstructured[pdf]
TextLoader	Plain .txt/.md	one doc per file	none
CSVLoader	CSV/TSV exports	one doc per row	none
JSONLoader	JSON / JSON lines	per jq match	jq (or jq-py)
DirectoryLoader	A folder tree	delegates to loader_cls	depends
WebBaseLoader	Public web pages	one doc per URL	beautifulsoup4
RecursiveUrlLoader	A site, crawled	one doc per URL	beautifulsoup4
YoutubeLoader	Video transcripts	one doc per video	youtube-transcript-api
UnstructuredWordDocumentLoader	.docx	element or file	unstructured[docx]
SQLDatabaseLoader	SQL query results	one doc per row	sqlalchemy

Check the loader's granularity before you write the splitter. Row-per-document loaders often need *no* splitting, while whole-file loaders always do.

5. Loading a PDF

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("reference-docs/langchain_demo.pdf")
documents = loader.load()

print(f"{len(documents)} documents (one per page)")
first = documents[0]
print(first.metadata)          # {'source': 'reference-docs/...', 'page': 0}
print(first.page_content[:300])
```

Sanity checks to run on any new corpus - they catch most ingestion bugs in under a minute:

```
empty = [d for d in documents if not d.page_content.strip()]
print("empty pages:", len(empty))          # >0 means scanned pages -> needs OCR

lengths = [len(d.page_content) for d in documents]
print("chars: min/avg/max =",
      min(lengths), sum(lengths) // len(lengths), max(lengths))

print("metadata keys:", set().union(*(d.metadata.keys() for d in documents)))
```

Choosing a PDF parser

PDF has no notion of reading order, so every parser guesses. On a two-column paper a naive parser interleaves the columns and produces text that is fluent nonsense - embeddings happily encode it and retrieval quietly degrades. Compare parsers on *your* documents, not on a benchmark.

Parser	Strengths	Weaknesses	Use when
PyPDFLoader (pypdf)	Pure Python, no system deps, fast enough	Weak on columns and tables	Simple, text-first reports
PyMuPDFLoader	Fast, better layout and coordinates	AGPL licence to check	Large corpora, mixed layouts
UnstructuredPDFLoader	Tables, titles, element types	Heavy deps, slow	Layout matters more than speed
OCR (pytesseract / RapidOCR)	Reads scans and images	Slow, error-prone	Pages that load empty

6. Loading directories, tabular data and the web

Directories

```
from langchain_community.document_loaders import DirectoryLoader, PyPDFLoader

loader = DirectoryLoader(
    "reference-docs/",
    glob="**/*.pdf",
    loader_cls=PyPDFLoader,
    show_progress=True,
    use_multithreading=True,
    silent_errors=True,          # skip unreadable files instead of aborting the run
)
docs = loader.load()
```

`silent_errors=True` is convenient and dangerous: a corrupt file disappears from the index without anyone noticing. Log what was skipped and assert on the expected document count.

CSV - one document per row

```
from langchain_community.document_loaders import CSVLoader

loader = CSVLoader(
    file_path="data/faq.csv",
    source_column="url",  # becomes metadata['source']
    metadata_columns=["category"],  # kept as filterable metadata
    csv_args={"delimiter": ",", "quotechar": "'"},
)
rows = loader.load()
# page_content looks like: "question: How do I reset?\nanswer: Use the link ..."
```

JSON - pick fields with a jq expression

```
from langchain_community.document_loaders import JSONLoader

def add_meta(record: dict, metadata: dict) -> dict:
    metadata["ticket_id"] = record.get("id")
    metadata["status"] = record.get("status")
    return metadata

loader = JSONLoader(
    file_path="data/tickets.json",
    jq_schema=".[]",
    content_key="body",
    metadata_func=add_meta,
)
```

Web pages

```
from langchain_community.document_loaders import WebBaseLoader

loader = WebBaseLoader(["https://python.langchain.com/docs/introduction/"])
loader.requests_per_second = 2  # be polite; respect robots.txt and ToS
pages = loader.load()
```

Scraped pages carry navigation, cookie banners and footers into your index. Either pass a BeautifulSoup `bs_kwargs` filter to keep only the main content, or clean the text before splitting (section 9).

7. load(), lazy_load(), aload()

Method	Returns	Memory	Use when
<code>load()</code>	<code>list[Document]</code>	whole corpus in RAM	Small sets, notebooks, tests
<code>lazy_load()</code>	<code>Iterator[Document]</code>	one document at a time	Large corpora, streaming into a store
<code>aload()</code>	awaitable list	whole corpus in RAM	Many network-bound sources in parallel

```
# Stream: load, split and index in batches - constant memory, resumable.
batch = []
for doc in loader.lazy_load():
    batch.extend(splitter.split_documents([doc]))
    if len(batch) >= 200:
        vectorstore.add_documents(batch)
        batch.clear()
if batch:
    vectorstore.add_documents(batch)
```

8. Splitting: the decision that drives retrieval quality

Chunking is where most retrieval quality is won or lost. Chunks that are too large dilute the embedding (one vector tries to represent five topics); chunks that are too small lose the context that makes an answer usable.

The default: RecursiveCharacterTextSplitter

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,          # characters, unless you use a token encoder
    chunk_overlap=150,        # ~10-20% of chunk_size
    separators=["\n\n", "\n", ". ", " ", ""], # tried in order
    add_start_index=True,     # metadata['start_index'] -> highlight in the source
)
chunks = splitter.split_documents(documents)
print(len(documents), "pages ->", len(chunks), "chunks")
```

It is "recursive" because it tries the first separator, and only if a piece is still too large does it fall back to the next one. That is why the separator order matters: paragraphs first, then lines, then sentences, then characters as a last resort.

Counting tokens, not characters

Embedding models have token limits, so character counts are a proxy. Split on the same tokenizer the model uses when the limit is tight:

```
splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
    encoding_name="cl100k_base", chunk_size=400, chunk_overlap=60
)
```

Splitting by structure

If the document has structure, split on it before falling back to characters - structure-aware chunks are almost always better neighbours:

```
from langchain_text_splitters import (
    MarkdownHeaderTextSplitter, RecursiveCharacterTextSplitter, Language
)

md = MarkdownHeaderTextSplitter(
    headers_to_split_on=[("#", "h1"), ("##", "h2"), ("###", "h3")]
)
# heading path is copied into each chunk's metadata

py = RecursiveCharacterTextSplitter.from_language(
    language=Language.PYTHON, chunk_size=800, chunk_overlap=100
)
# splits on class/def boundaries
```

Starting points by content type

Content	chunk_size	overlap	Why
Prose, reports, papers	800-1200 chars	10-15%	About a paragraph; one idea per vector
Slides, FAQs, tickets	300-600 chars	0-10%	Already short and self-contained
Code	600-1000 chars	~10%	Split on function boundaries, not lines
Legal / dense reference	1200-2000 chars	15-20%	Clauses lose meaning out of context

Treat these as the first experiment, not the answer. Build a small evaluation set of real questions with known correct sources, then measure recall@k while you vary chunk size - it is the cheapest quality win in the whole pipeline.

9. Metadata, cleaning and citations

Enrich while you still have the source

```
import hashlib
from datetime import datetime, timezone
```

```
for doc in documents:
    doc.metadata["doc_hash"] = hashlib.sha256(
        doc.page_content.encode("utf-8")
    ).hexdigest()[:16]
    doc.metadata["ingested_at"] = datetime.now(timezone.utc).isoformat()
    doc.metadata["collection"] = "reference-docs"
```

The hash gives you cheap deduplication and incremental re-indexing: on the next run, skip any chunk whose hash is already stored, and delete stored chunks whose source file no longer produces them.

Clean the text before splitting

```
import re

def clean(text: str) -> str:
    text = re.sub(r"-\\n(?:\\w)", "", text)      # rejoin hyphenated line breaks
    text = re.sub(r"\\n{3,}", "\\n\\n", text)    # collapse blank-line runs
    text = re.sub(r"[ \\t]{2,}", " ", text)      # collapse runs of spaces
    return text.strip()

for doc in documents:
    doc.page_content = clean(doc.page_content)
```

Do not strip too eagerly. Removing all newlines destroys the paragraph boundaries the splitter relies on, and removing repeated headers by regex can silently delete real content. Clean, then re-read a few pages.

Carry the citation into the prompt

```
def format_docs(docs):
    return "\\n\\n".join(
        f"[{d.metadata.get('source')} p.{d.metadata.get('page', '?')}]\\n{d.page_content}"
        for d in docs
    )
```

If the citation is not in the context string, the model cannot cite it - it will invent a plausible page number instead. Formatting the source into the text is the single most effective anti-hallucination measure in a RAG prompt.

10. Error handling and robustness

```
import logging
from pathlib import Path

from langchain_community.document_loaders import PyPDFLoader

log = logging.getLogger(__name__)

def load_folder(folder: str) -> tuple[list, list[str]]:
    docs, failed = [], []
    for path in sorted(Path(folder).rglob("*.pdf")):
        try:
            loaded = PyPDFLoader(str(path)).load()
        except Exception as exc:      # corrupt, encrypted, truncated
            log.warning("skipping %s: %s", path, exc)
            failed.append(str(path))
            continue
        if not any(d.page_content.strip() for d in loaded):
            log.warning("%s has no extractable text (scanned?)", path)
            failed.append(str(path))
            continue
        docs.extend(loaded)
    return docs, failed
```

Failure modes worth handling explicitly:

- **Encrypted or password-protected PDFs** - raise, or pass the password.
- **Scanned pages** - load successfully but return empty text. Detect with an emptiness check and route to OCR.
- **Network sources** - timeouts and rate limits; retry with backoff, and cache raw responses so a re-run does not re-download.
- **Encoding errors in text/CSV** - pass autodetect_encoding=True or specify the encoding.
- **Partial failure** - decide up front whether a failed file should abort the run or be reported. Both are valid; silence is not.

11. End-to-end: from this PDF to an answer

The whole pipeline, in the shape you will use in the session. It reads the provider from .env so it works with whichever free API key you set up.

```
import os
from dotenv import load_dotenv

from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings
from langchain.chat_models import init_chat_model
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

load_dotenv()

# 1. Load -----
docs = PyPDFLoader("reference-docs/langchain_demo.pdf").load()

# 2. Split -----
chunks = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=150, add_start_index=True
).split_documents(docs)

# 3. Embed + store -----
store = Chroma.from_documents(
    chunks,
    OpenAIEmbeddings(model="text-embedding-3-small"),
    collection_name="handout",
    persist_directory=".chroma",
)

# 4. Retrieve -----
retriever = store.as_retriever(search_type="mmr", search_kwargs={"k": 4})

# 5. Generate -----
prompt = ChatPromptTemplate.from_template(
    "Answer the question using only the context below. "
    "If the context does not contain the answer, say you do not know. "
    "Cite the page for every claim.\n\n"
    "Context:\n{context}\n\nQuestion: {question}"
)

llm = init_chat_model(
    os.getenv("LLM_MODEL", "gpt-4o-mini"),
    model_provider=os.getenv("LLM_PROVIDER", "openai"),
)

def format_docs(docs):
    return "\n\n".join(
```

```

        f"[p.{d.metadata.get('page')}] {d.page_content}" for d in docs
    )

chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)

print(chain.invoke("What chunk size should I use for dense legal text, and why?"))

```

search_type="mmr" (maximal marginal relevance) trades a little relevance for diversity, which reduces the classic failure where the top four chunks are four near-copies of the same paragraph.

12. Taking ingestion to production

- **Make ingestion idempotent.** Re-running the job must not duplicate chunks. Key on a stable id (source path + chunk hash), or use LangChain's indexing API with a record manager to handle inserts, updates and deletes.
- **Re-index incrementally.** Embedding is the expensive step; only embed chunks whose hash changed. A nightly full re-embed of a large corpus is a bill, not a design.
- **Version the pipeline.** Store the loader, splitter settings and embedding model name in each chunk's metadata. When you change the embedding model you must re-embed everything - you will want to know what is stale.
- **Never mix embedding models in one collection.** Vectors from different models are not comparable; the search will still return results, just meaningless ones.
- **Separate collections per tenant or per permission scope,** or store the scope as metadata and always filter on it. Retrieval that ignores permissions is a data leak with extra steps.
- **Log and measure.** Documents loaded, files skipped, chunks written, tokens embedded, wall time. Without these, a corpus that silently halves is invisible until users complain.
- **Evaluate retrieval separately from generation.** When an answer is wrong, first check whether the right chunk was retrieved at all; that tells you which half of the system to fix.

13. Troubleshooting

Symptom	Likely cause	Fix
Documents load but text is empty	Scanned/image-only PDF	Run OCR (pytesseract, RapidOCR) before indexing
Text is jumbled or interleaved	Multi-column layout, naive parser	Try PyMuPDF or Unstructured; verify by eye
Retrieval returns nothing relevant	Chunks too large, or query/doc mismatch	Reduce chunk size; test recall@k on real questions
Answers cite wrong page	Metadata lost during splitting/formatting	Keep metadata; format source into the context
Re-run duplicates everything	No stable ids	Hash content; use the indexing API
Ingestion runs out of memory	load() on a large corpus	Switch to lazy_load() and batch writes
Vector store rejects metadata	Nested dict or list values	Flatten to scalars before writing

14. Exercises

- 1 Load this file with PyPDFLoader and print the number of pages, the metadata keys and the character count per page.
- 2 Split it at chunk_size 300, 1000 and 2000 with the same overlap ratio. Ask the same question of each index and compare which chunks come back.
- 3 Load the same file with a second parser (PyMuPDF or Unstructured) and diff the extracted text of page 2 against PyPDFLoader's.
- 4 Add doc_hash to every chunk, re-run ingestion, and verify that nothing is written the second time.
- 5 Break it on purpose: truncate a copy of the PDF, point the loader at the folder, and make the job report the failure instead of skipping it silently.

Appendix A. Sample data for chunking tests

The passages below are deliberately short, self-contained and slightly repetitive - useful for watching how chunk boundaries and overlap behave, and for checking that retrieval returns the passage you expect.

A.1 Pangram

The quick brown fox jumps over the lazy dog. This sentence contains every letter of the alphabet and is commonly used for testing text extraction and font rendering.

A.2 Project description

LangChain was created to simplify the development of applications powered by language models. It provides modular components for building chains, agents and retrieval systems, and it standardises the interfaces between them so that a model, a vector store or a loader can be swapped without rewriting the application.

A.3 Feature list

- Prompt templates for structured inputs
- Output parsers for structured outputs
- Chains for sequential operations
- Agents for autonomous, tool-using decision making
- Memory for conversation context
- Vector stores for semantic search
- Callbacks and tracing for observability

A.4 Question set for retrieval tests

Each question below is answerable from exactly one section of this handout, which makes it usable as a tiny evaluation set: record the section you expect, then check whether it appears in the top-k retrieved chunks.

Question	Expected source
What overlap should I use for legal text?	Section 8, table
Why does a page load with no text?	Sections 10 and 13
How do I avoid duplicating chunks on re-run?	Sections 9 and 12

Question	Expected source
When should I use <code>lazy_load()</code> ?	Section 7
Which parser handles two-column papers?	Section 5, table

Appendix B. Further reading

- LangChain documentation - Document loaders and text splitters: `python.langchain.com/docs/concepts/document_loaders`
- LangChain indexing API (record manager) for incremental re-indexing.
- The vector store's own docs on metadata filtering - the limits differ per store.
- Repository companions: `research-papers/` for the underlying RAG papers, and the session folders for runnable versions of the code in this handout.

Handout for the agent-to-production course. Code targets LangChain 0.3.x; APIs move quickly, so check the current docs when something does not import.